

Location Retrieval LOC3X1 Process: Camel Route Handbook

Alberto Espinosa
KSquare Group

November 1, 2025

Abstract

This document presents a comprehensive technical overview and developer handbook for the “loc3x1-camel-route” project. It explains the technology stack, architecture, components, configuration, and data flows implemented with Spring Boot and Apache Camel (YAML DSL). It also covers the runtime requirements, how to build and run the application, how to validate core functionality, and the interactions with external services used for enrichment and mapping. The objective is to enable any technical developer to understand, maintain, and extend the project at any time.

1 Introduction

The project implements the Location Retrieval LOC3X1 business process and related capabilities using Apache Camel integrated in a Spring Boot application. Routes are primarily authored in YAML DSL and exposed via Spring REST controllers that orchestrate “direct:” endpoints. Additional services perform validation, data enrichment against external SOAP endpoints, mapping to canonical XSD-based DTOs, and rule-based transformations (e.g., Fire District handling).

2 Technology Stack and Requirements

2.1 Languages and Runtimes

- Java 17 (source compatibility). Tested running on Java 25 runtime.
- Spring Boot 3.3.0.

- Apache Camel 4.14.0 with YAML DSL, timer, and log components.

2.2 Libraries and Frameworks

- Camel Spring Boot Starter and YAML DSL for route definitions.
- Jakarta XML Bind (JAXB) and JAX-WS APIs; Glassfish JAXB runtime.
- Jackson XML; Freemarker for templating (support utilities).
- Apache HttpClient for HTTP/SOAP calls to external services.
- MapStruct (mapping support) and Lombok (data classes).
- Local JARs: `libs/xsd-javapojos-0.0.3-SNAPSHOT.jar`, `libs/soap-clients-0.0.1-SNAPSHOT.jar`

2.3 Build Systems

Both Gradle and Maven files are present. In environments where the Gradle wrapper is incomplete or fails due to JDK compatibility, Maven can be used. The project pom is configured with all required dependencies, including the local JARs (system scope).

2.4 External Services and Ports

Several enrichment and mapping steps call local mock SOAP endpoints at `http://localhost:8085`. Configuration for service calls in the YAML routes also references hosts/ports for `LOC3X1M`, `CRP11X1`, and `CRP10X1`. These should be available when running full end-to-end flows.

3 Project Structure

3.1 Top-Level

- `src/main/java`: Java sources (controllers, services, model, processors).
- `src/main/resources`: `application.yml`, YAML route files, templates.
- `libs`: local binary dependencies required by business logic.
- `build.gradle`, `pom.xml`: Gradle/Maven build descriptors.

3.2 Key Packages

- `com.locationretrievalloc3x1process.webservice.controller`: Spring MVC controllers exposing REST endpoints and bridging to Camel “direct:” routes.
- `com.locationretrievalloc3x1process.webservice.service`: Validation, enrichment (state/country), mapping, RAND integration, fire district rules, and XML parsing utilities.
- `com.locationretrievalloc3x1process.webservice.locationretrievalloc3x1process.com`: Camel processors and route configuration helpers.
- `com.chubb.converter.util` and `com.chubb.mappers.xmldto`: XML/XPath parsing and DTO classes from local JARs.

4 Configuration

4.1 Application Configuration

`src/main/resources/application.yml`:

- Camel YAML routes enabled with locations: `classpath:camel/*.yaml`, `classpath:*.yaml` to load both root-level and `camel/` folder YAML routes.
- Camel servlet mapping: `/api/*` (for potential REST DSL usage).
- Host/port placeholders: `loc3x1m.host/port`, `crp11x1.host/port`, `crp10x1.host/port`.

5 Runtime Architecture

5.1 Entry Points

The Spring Boot application main class is `LocationRetrievalApplication`. It scans the `com.locationretrievalloc3x1process.webservice` base package, wiring controllers and services.

5.2 Controllers and Endpoints

- GET `/api/index`: simple health-style endpoint executing a dynamic client from `soap-clients` (returns a greeting string).

- `POST /location/retrieveList3x1`: accepts a `JSON LocationRequest`, sends to Camel `direct:locationProcess`.
- `POST /location/retrieveWithTaxingJurisdictions`: accepts a `JSON LocationRequest`, sends to Camel `direct:locationWithTaxingJurisdictionsProcess`.
- `POST /location/loc3x1`: accepts XML, parses to `LocationRequest`, and returns LOC3X1 XML transformed from RAND response.

5.3 Camel Routes (YAML DSL)

Key route files:

- `camel/loc3x1-rest.yaml`: Defines `direct:locationProcess`, `direct:locationWithTaxingJurisdictionsProcess`, and `direct:createSimpleFault` routes. Implements validation via `ValidationService`, optional enrichment via `EnrichmentService`, mapping via `MappingService`, and Fire District checks via `FireDistrictService`. Faults are produced by `SimpleFaultService`.
- `loc3x1-routes.yaml`: Alternative route set that orchestrates calls to LOC3X1M and corporate services (CRP11X1, CRP10X1), including `doTry/doCatch` blocks and global exception handling.
- `routes.yaml`: A simple timer-based “Hello World” example route.

5.4 Services and Processing

- **ValidationService**: Validates request fields. Taxing jurisdictions flow requires `postalCode` and `state/province` indicators.
- **EnrichmentService**: Optionally queries mock SOAP services to enrich state or country data using XPath/XML parsing helpers.
- **MappingService**: Produces `LocationListReply` via `LocationListReplyService` and RAND integration in `RansService`.
- **FireDistrictService**: Normalizes and warns on invalid FireDistrict codes (e.g., “9999”) by updating reply status and error list.
- **XmlParsingService**: JAXB-based XML to object conversion utility.
- **MyCustomProcessor**: Example processor used in the hello/timer route.

6 Data Flow

6.1 Retrieve List (3x1)

1. Controller receives `LocationRequest` JSON and sends to `direct:locationProcess`.
2. Route validates the request; on success, may enrich missing state/country values.
3. Mapping service transforms the request to a canonical reply, optionally calling `RAND` and `XML` mappers.
4. Fire District checks apply business rules; response is returned to the controller as JSON.
5. On invalid input, route returns a `SimpleFault` JSON payload.

6.2 Retrieve With Taxing Jurisdictions

Similar to the list flow but requires stricter validation and produces a reply including jurisdiction data. Faults are returned when required fields are missing.

7 How to Build and Run

7.1 Prerequisites

- Java 17+ (tested with Java 25 runtime).
- Maven 3.9+ installed. Gradle can be used when properly configured, but Maven is recommended in this environment.
- Optional: local mock services running on `localhost:8085` for full enrichment and mapping flows.

7.2 Run

From the project root:

```
mvn -DskipTests spring-boot:run
```

The application starts on port 8080. Camel reports started routes, including `direct:locationProcess`, `direct:locationWithTaxingJurisdictionsProcess`, and `direct:createSimpleFault`.

7.3 Quick Verification

- Health-style endpoint:

```
curl http://localhost:8080/api/index
```

- Invalid request (fault path):

```
curl -X POST http://localhost:8080/location/retrieveList3x1 \
-H 'Content-Type: application/json' \
-d '{"addressLine1":"123 Main St","cityName":"Dallas"}'
```

8 Configuration Notes

- Update `application.yml` `camel.springboot.yaml-routes.locations` if routes are moved or new folders are added.
- Set `loc3x1m`, `crp11x1`, and `crp10x1` host/port values to point at real services when not using local mocks.
- Ensure `libs/` JARs are present; Maven includes them with system scope.

9 Testing and Validation

- Unit tests can be added around service methods (validation, enrichment decisions, mapping logic, Fire District rules).
- Integration tests can use WireMock (already a test dependency in Gradle) to simulate external SOAP endpoints.
- For XML flows, validate XPath parsing and marshalling/unmarshalling against sample payloads in `templates/`.

10 Maintenance and Extension

- New routes: author in YAML under `src/main/resources/camel/` and include in locations.
- Add processors/services for transformations; prefer small, cohesive classes.

- Keep external service contracts in local JARs or Maven artifacts; document expected endpoints and payloads.
- Monitor Camel and Spring Boot versions for compatibility with your runtime JDK.

11 Conclusion

This project demonstrates a clean integration of Apache Camel YAML routes within a Spring Boot service exposing REST endpoints. The controllers bridge HTTP APIs to Camel “direct:” routes, enabling validation, enrichment, mapping, and business rule application. With this handbook, developers can quickly run, verify, and extend the service for LOC3X1 and related processes.